NOVA IMS INFORMATION MANAGEMENT SCHOOL

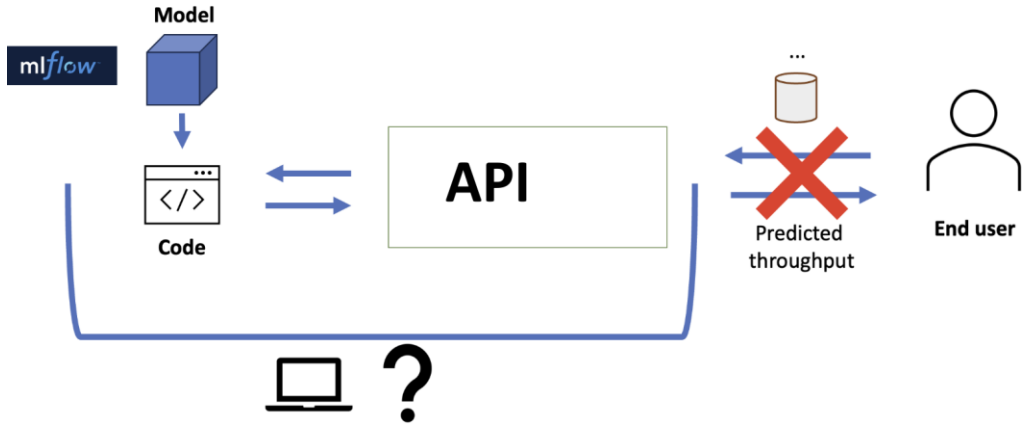
03:Model Deployment and Serving

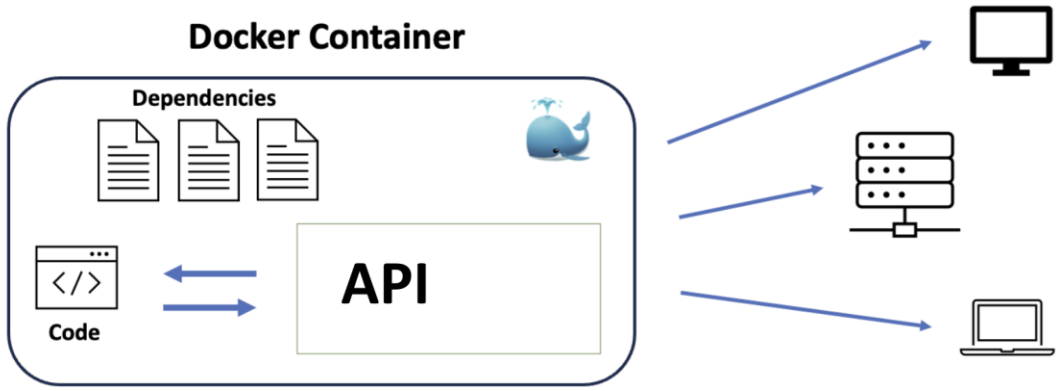
Nuno Rosa
nagostinho@novaims.unl.pt

Last week continuation and recap

03: Model Deployment Requirements

Containerization (Recap)

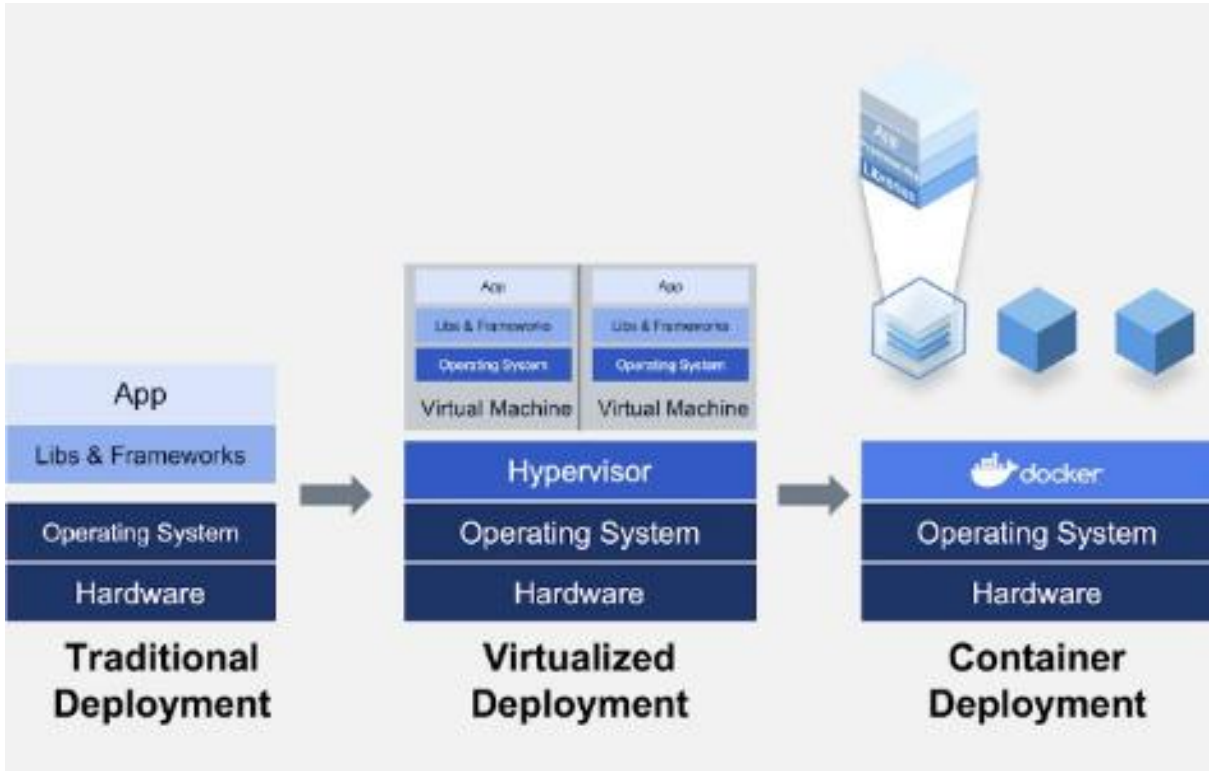




- We are not going to serve from our local computers
- We need to deploy the model in other platforms to serve
- The best solution in terms of flexibility and isolations is a container.

03: Model Deployment Requirements

Containerization (Recap)



- **Virtual machines access the hardware of a physical machine through a hypervisor**. The hypervisor creates an abstraction layer allowing the VM to access CPU, memory, and storage.
- **Containers represent a package that includes an executable with the dependencies it needs to run**. Each container shares the physical machine's hardware and operating system kernel with other containers.
- **Virtual machines are typically more resource-intensive than containers and less lightweight and portable**. Containers are a good choice for applications that need to be deployed quickly and optimized.

03: Model Deployment Requirements

Containerization (Recap)

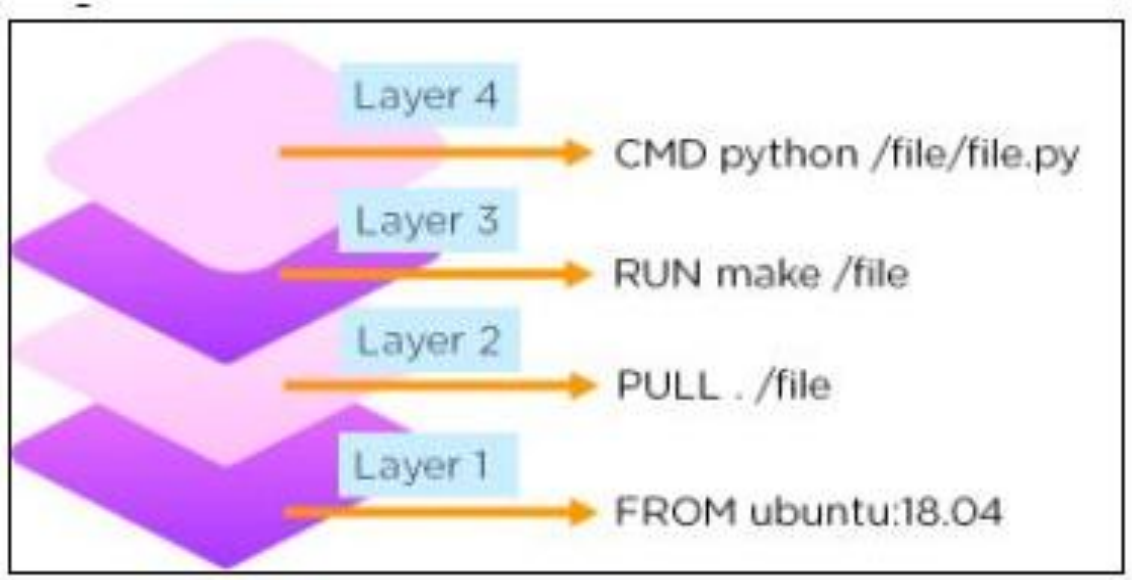
Feature	Container	Virtual machine
Operating system	Shares the host operating system's kernel	Has its own kernel
Portability	More portable	Less portable
Speed	Faster to start up and shut down	Slower to start up and shut down
Resource usage	Uses fewer resources	Uses more resources
Use cases	Good for portable and scalable applications	Good for isolated applications

- Containers are good for Web services, microservices (local architecture), cloud computing and to Continuous Integration and Delivery.
- Virtual machines are good to create full isolated environments such as to testing a new software in a safely sandboxed environment, to the development of models, test their software in different operating systems. They are also good option for public clouds and hybrid solutions.

03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
  - **Use Docker oficial images**. If you build containers on top of existent images, use trusted providers. Check the image source *https://hub.docker.com/ /python* .
  - **Use .dockerignore to prevent undesirable files and directories** from being contained in the image when we run the docker build command.
  - Take advantage of Docker caches, the layers. Layers are used to build Docker images.



- When building an image, Docker steps through the instructions in your Dockerfile, executing each in the order specified.
- As each instruction is examined, Docker looks for an existing image in its cache that it can reuse, rather than creating a new, duplicate image.
- If you have several layers, then you can order them from the less frequently changed to the more frequently changed.

03: Model Deployment Requirements

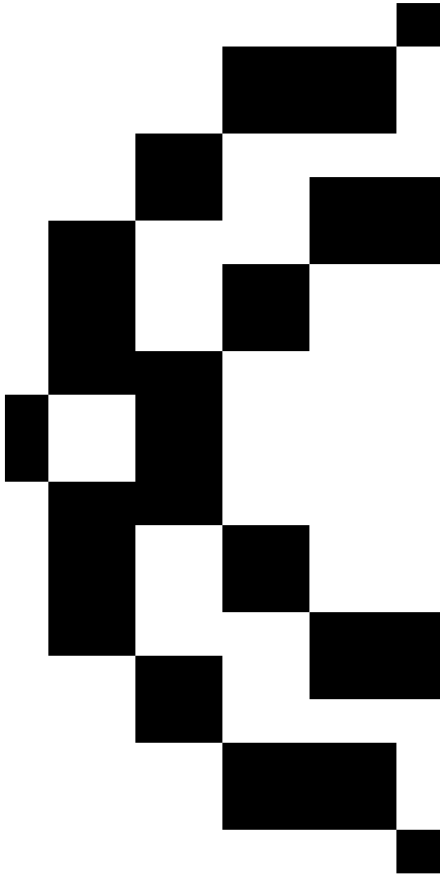
Docker good practices

- There are a series of good practices to implement each relevant component of Docker:

  - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**

```
# train/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY train.py requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "train.py"]

# serve/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY serve.py requirements.txt model.pkl ./
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "serve.py"]
```



03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
  - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**
  - **A separate container for each service allows you to grow one of your web servers horizontally** to accommodate the extra traffic.

Docker Compose is a tool that lets you define and run multi-container Docker applications with a simple YAML file

```
version: "3.8"
services:
  api:
    image: myregistry.com/ml-api:1.2.0
    ports:
      - "5000:5000"
    deploy:
      replicas: 4   #means running 4 instances of the same containerized service.

  redis:
    image: redis:6.2-alpine

  store:
    image: myregistry.com/feature-store:2.0.1
```

03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:

  - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**
  - **A separate container for each service allows you to grow one of your web servers horizontally** to accommodate the extra traffic.
  - There are tags that you can use for the basic image of the container. **Avoid utilizing these tags for deployment containers since these tags will get changed often**, and it might lead to discrepancies in the production environment, e.g. **image: example.com/ml-api:latest**
  - **Use unique tags for deployments**. You can extend your production cluster to numerous nodes with ease. It prevents inconsistencies, and hosts will not fetch any other docker image version.

REPOSITORY	TAG	IMAGE ID
myapp	debug	26eef669f4af
testbuild	latest	26eef669f4af
ubuntu	16.04	005d2078bdfa
ubuntu	latest	1d622ef86b13

One can have repos with same name but different tags

03: Model Deployment Requirements

Containerization

What if ...

- When a model is deployed in several copies, how can the workload be balanced?
- What happens if the model becomes unresponsive, for example, if the machine hosting it fails? How can that be detected and a container reprovisioned?
- How can a model running on multiple machines be upgraded, with assurances that old and new versions are switched on and off, and that the load balancer is updated with a correct sequence?

Solution

- Kubernetes, open source platform for container orchestration.
- No code for steps to set up, monitor, upgrand and stop. Just an API and a configurarion file where the users specify the desired state.
- Easy scalability: models are basically formulas that can be replicated in new machines, and Kubernetes handles its balancing and syncronization framework.

Model Deployment

Kubernetes

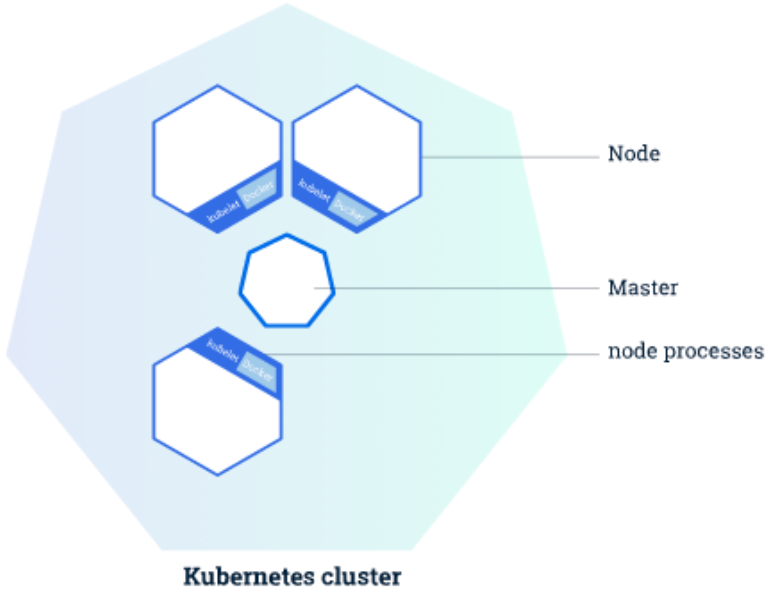
NOVA IMS INFORMATION MANAGEMENT SCHOOL

03: Model Deployment Requirements

Kubernetes

A Kubernetes cluster has two resources:

- **Master**: coordinator of the cluster.
- **Nodes**: workers that are essentially VMs or computers that run containers.



Relevant concepts:

- **kubelet**: is an agent or program which runs on each node, being responsible for the communication between the Kubernetes control plane and the nodes where the actual workload runs.
- **kubectl**: is the command line tool for communicating with a Kubernetes cluster's control plane, using the Kubernetes API,
  kubectl [command] [TYPE] [NAME] [flags]

Each node runs kubelet that manages it and communicated with the master. The deploy sequence essentially involves the following:

- kubectl tells the master to start application containers.
- Master schedules containers to run.

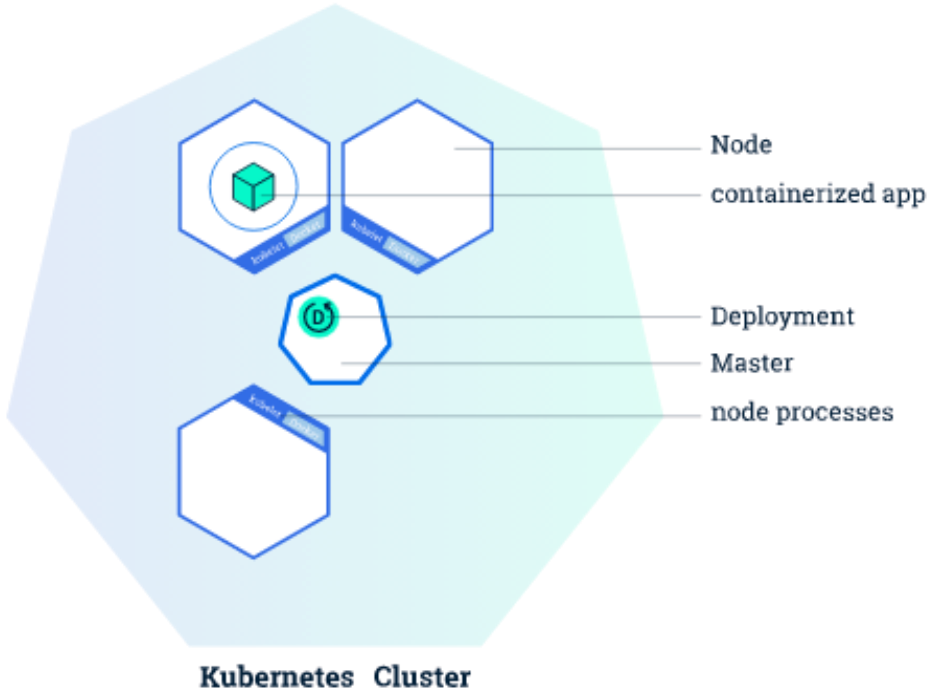
03: Model Deployment Requirements

Kubernetes

- Once we have a cluster running, we can deploy containerized applications (one or many) using a certain configuration.
- **This configuration is a set of instructions to Kubernetes to set up the application on the cluste**r, where we map each container to the nodes.

yml file to deploy

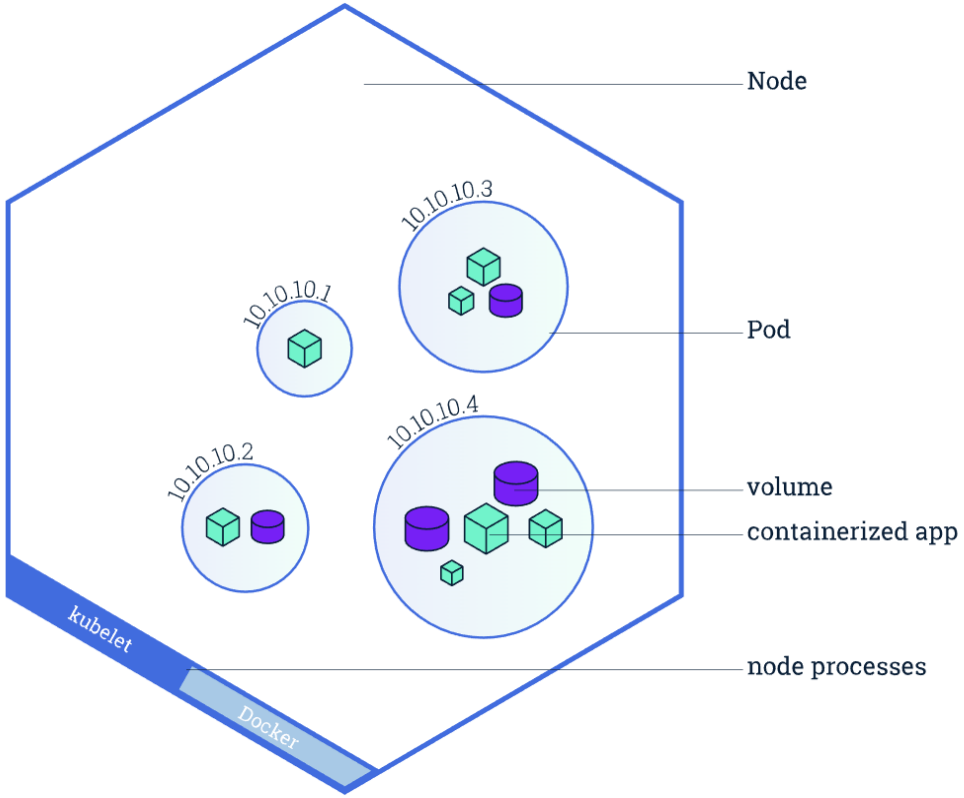
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-app-deployment
  labels:
    app: notes-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: notes-app
  template:
    metadata:
      labels:
        app: notes-app
    spec:
      containers:
      - name: notes-app-deployment
        image: pavansa/notes-app
        resources:
          requests:
            cpu: "100m"
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
```



- When creating a deployment, **we need to specify container images as well as the number of copies of each image we want to instantiate**.
- kubectl that uses an API to interact with the Master, will be responsible for the deployment.
- **Replicas are clones that facilitate self-healing for pods.** As with most processes and services, pods are liable to failure, errors, evictions, and deletion.

03: Model Deployment Requirements

Kubernetes



When an app is deployed as a container, it is encapsulated in a pod on a specific node. **Pods are the most basic unit in Kubernetes**. It is pods that are created and destroyed, not individual containers. One node can have several pods.

A pod is a collection of containers that share structures and can be related between each other:

- ❑ **Storage** (volumes)
- ❑ **Networking and IP address**

These related containers can be a web server and a database for instance.

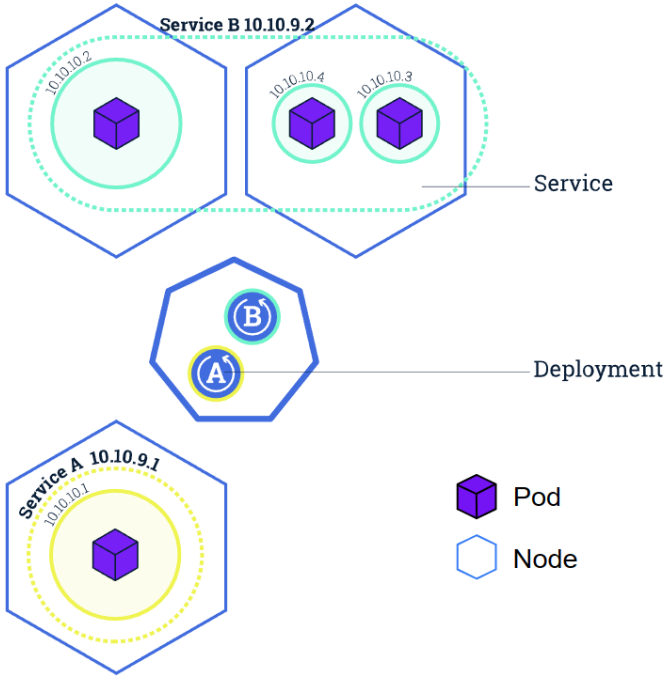
We can also have a single container in a pod.

03: Model Deployment Requirements

Kubernetes

To deploy your application, you need to expose the application to the outside world!

- A Kubernetes Service is an abstraction layer which defines a logical set of Pods and enables external traffic exposure, load balancing and service discovery for those pods.
- A Service is defined using YAML or JSON and lets pods get deleted and replicated in the cluster with no influence on our app. Couplings between pods.
- The pods with IPs cannot be typically accessed from the outside. Instead, a service can be used to allow external connections.



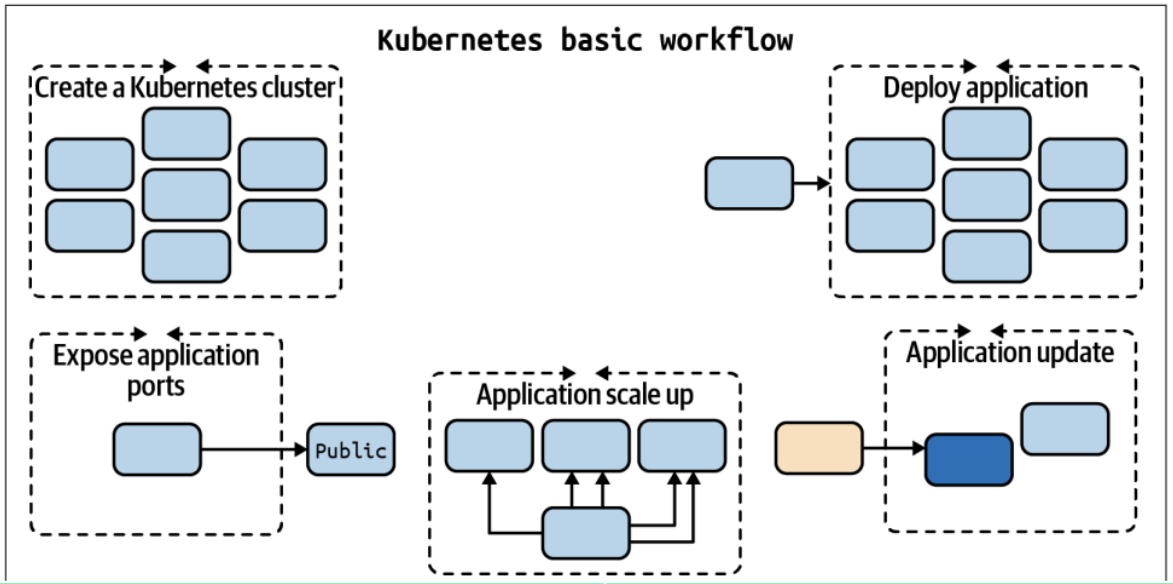
yml file for service

```yaml
apiVersion: v1
# Indicates this as a service
kind: Service
metadata:
 # Service name
 name: notes-app-deployment
spec:
 selector:
   # Selector for Pods
   app: notes-app
 ports:
   # Port Map
 - port: 80
   targetPort: 3000
   protocol: TCP
 type: LoadBalancer
```

Service Type	Use Case	Accessibility	Resource Allocation
ClusterIP	Internal communication between application components	**Within the cluster only**	Minimal resources needed
NodePort	External accessibility for web applications or APIs	Accessible from outside the cluster via a high-numbered port on the node	Additional resources needed
LoadBalancer	**Production environments** with high traffic volumes	Accessible from outside the cluster via a load balancer	Significant resources needed
Cloud Provider's Load Balancer	Using a cloud provider for Kubernetes	Accessible from outside the cluster via the cloud provider's load balancer	May result in cost savings and better performance

03: Model Deployment Requirements

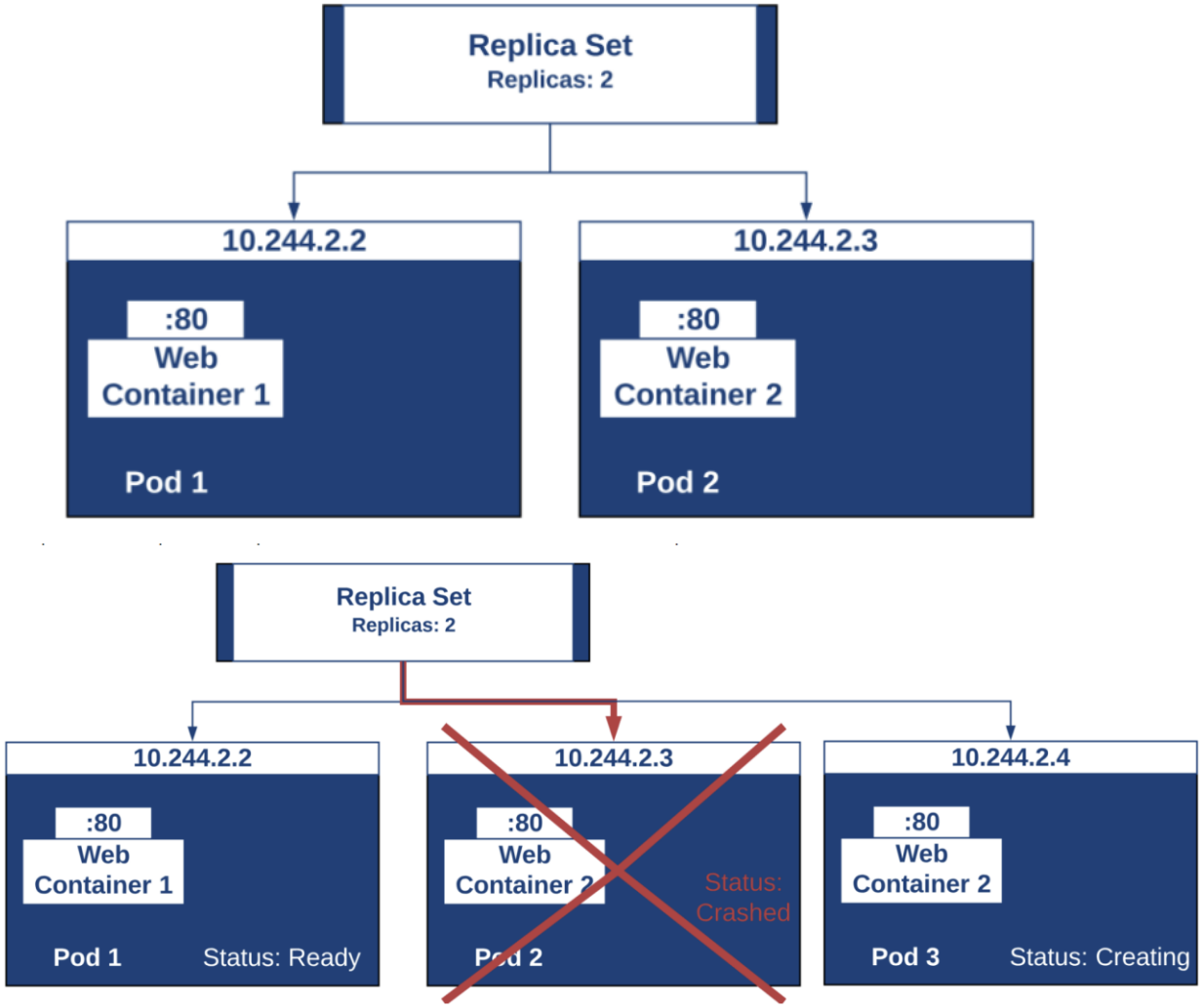
kubernetes final resume



Value of Pod	Description
Pending	The Pod has been accepted by the Kubernetes cluster, but one or more of the containers has not been set up and made ready to run. This includes time a Pod spends waiting to be scheduled as well as the time spent downloading container images over the network.
Running	The Pod has been bound to a node, and all of the containers have been created. At least one container is still running, or is in the process of starting or restarting.
Succeeded	All containers in the Pod have terminated in success, and will not be restarted.
Failed	All containers in the Pod have terminated, and at least one container has terminated in failure. That is, the container either exited with non-zero status or was terminated by the system.
Unknown	For some reason the state of the Pod could not be obtained. This phase typically occurs due to an error in communicating with the node where the Pod should be running.

03: Model Deployment Requirements

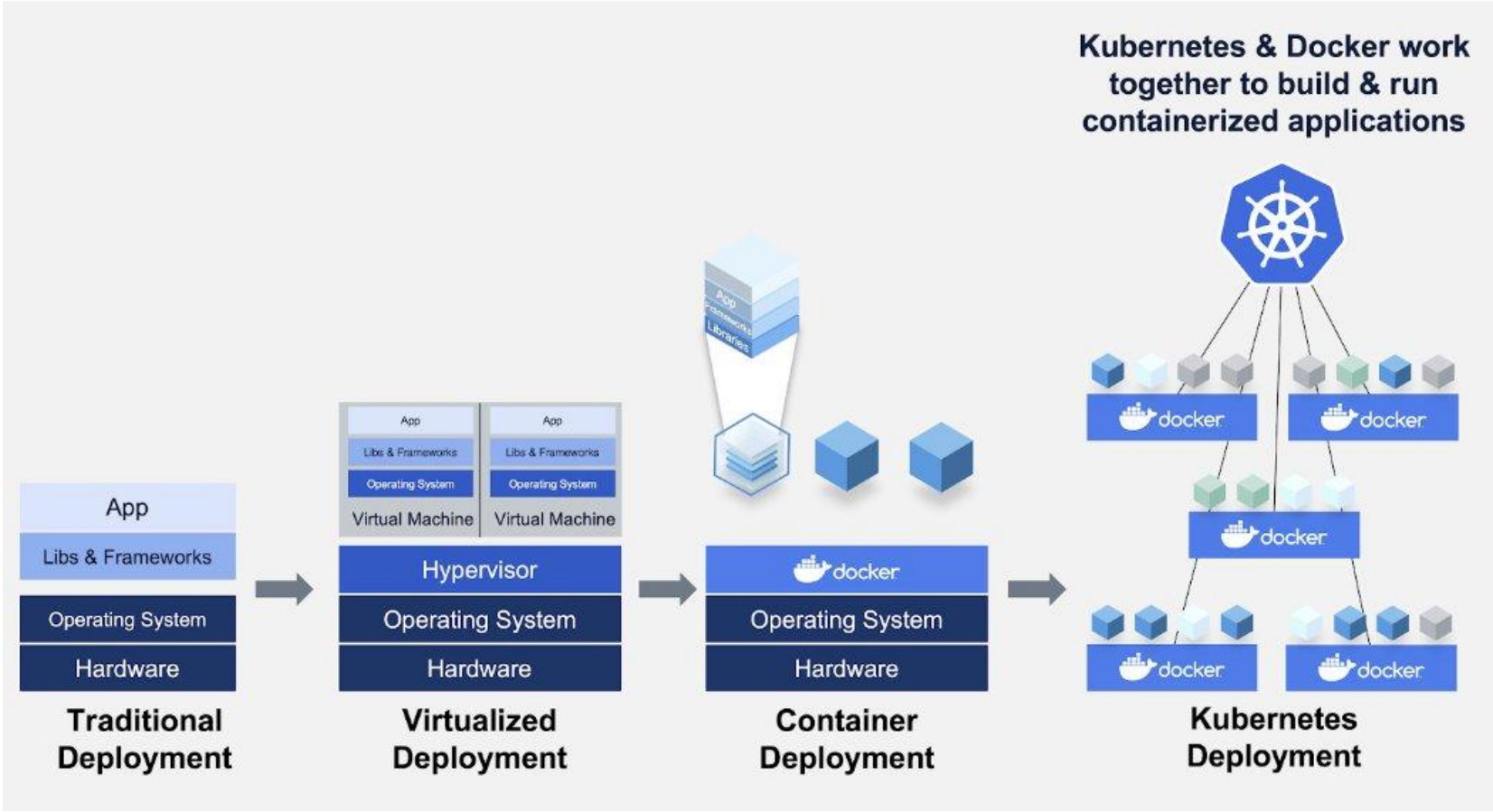
kubernetes final resume



- **ReplicaSet** ensures that a specified number of replicas of a pod are always running. It manages the lifecycle of pods and maintains the desired state of the application.
- They are controlled by **Deployment**, a Kubernetes object that manages a set of identical pods.
- Deployment enables automated easy rollbacks to previous versions by creating a replica set and updating it with the new configuration.

03: Model Deployment Requirements

kubernetes final resume

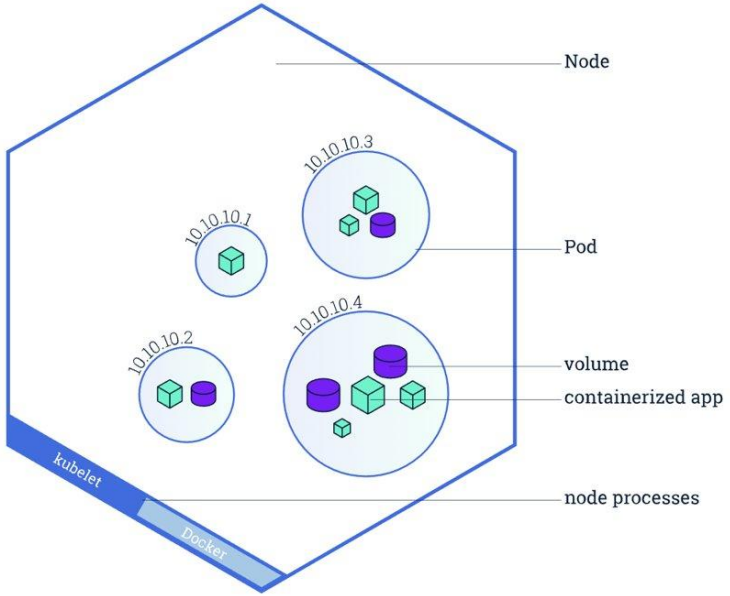


Trade-off between complexity and the problem we want to solve

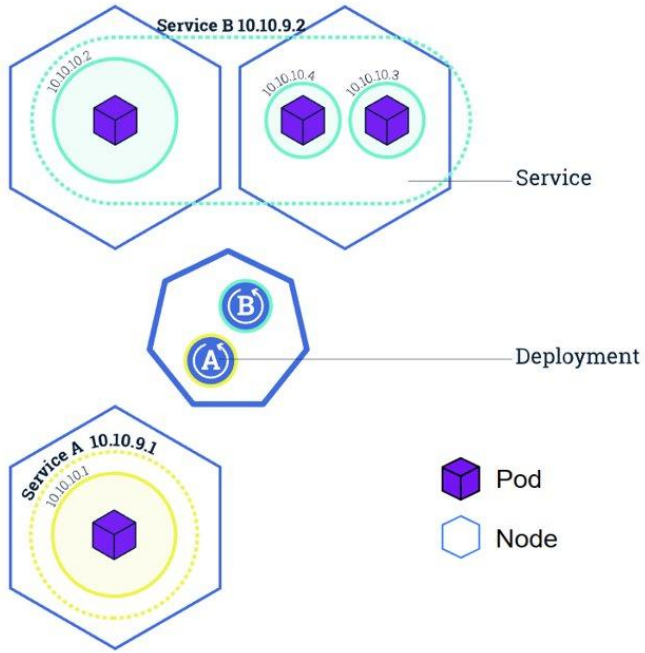
03: Model Deployment Requirements

kubernetes final resume

Kubernetes is an orchestration system for containers which is responsible of coordinating clusters of nodes at scale, in production



- The **basic unit structure of pods**.
- We see pods as scheduling units, containing one or more containers.
- **A container image is a lightweight, standalone, executable package of a piece of software**.
- These pods are distributed across hosts in a cluster to provide high availability service.

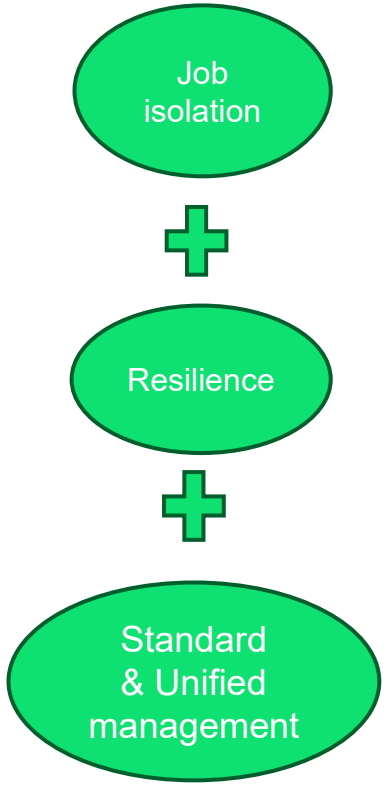


Docker and Kubernetes **are not competitors**! Docker helps on the creation of containers.

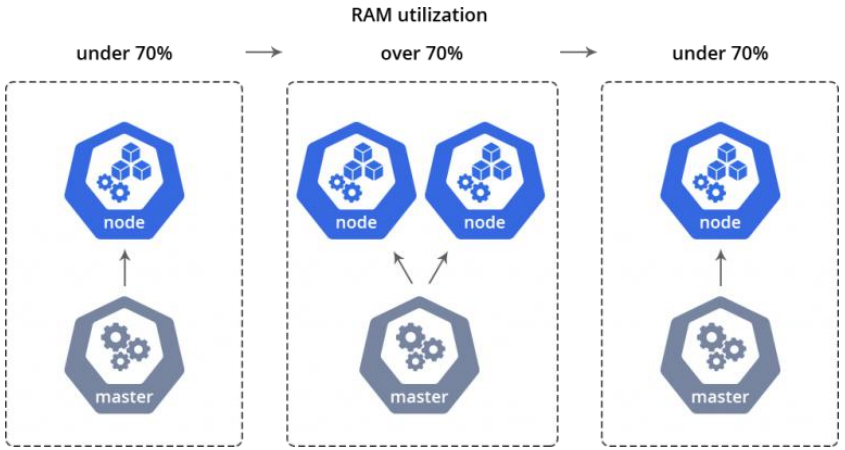
03: Model Deployment Requirements

kubernetes final resume

What problems Kubernetes try to solve?



A mature organization should desire:



Scalability: is not about "adding more hardware," in many cases it's about the underlying distributed systems architecture.

Main advantages:

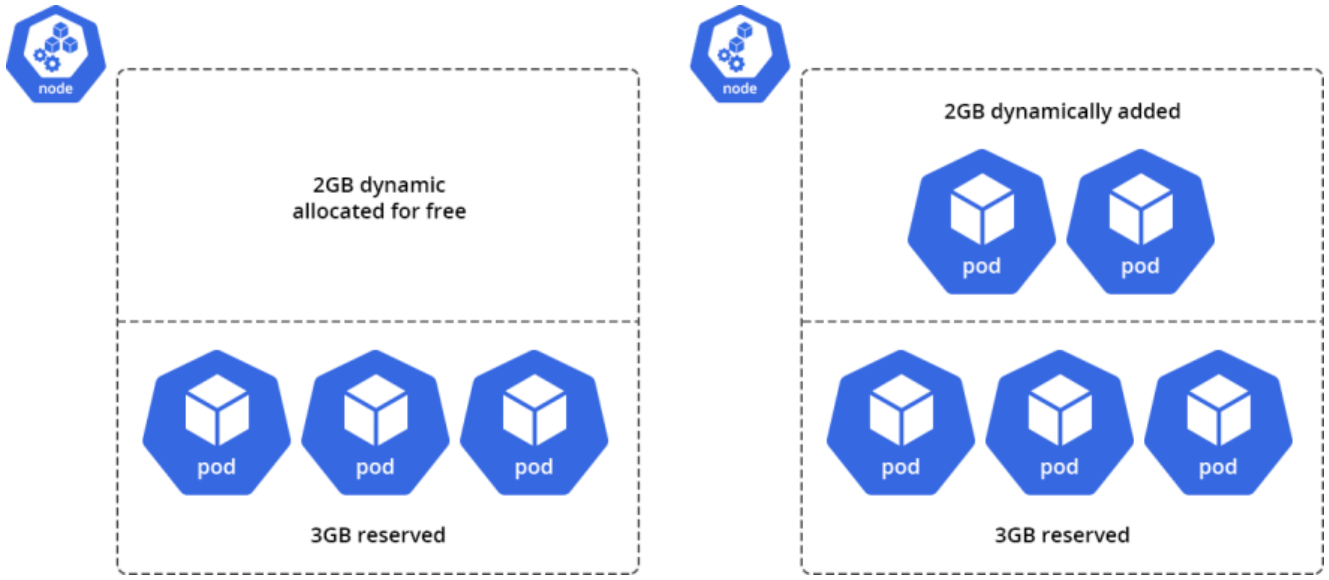
- ❑ Switch from a single machine to a distributed cluster.
- ❑ Different stages of your machine learning pipeline can request different amounts or types of resources.
  - Data preparation step may benefit more from running on multiple machines.
  - Model training may benefit more from computing on top of GPUs or tensor processing units (TPUs).

03: Model Deployment Requirements

kubernetes final resume

Why scalability is so relevant?

Business wants to grow and pay less... DevOps, MLOps want a stable platform that can run applications at scale...



Vertical scaling in a cloud system:

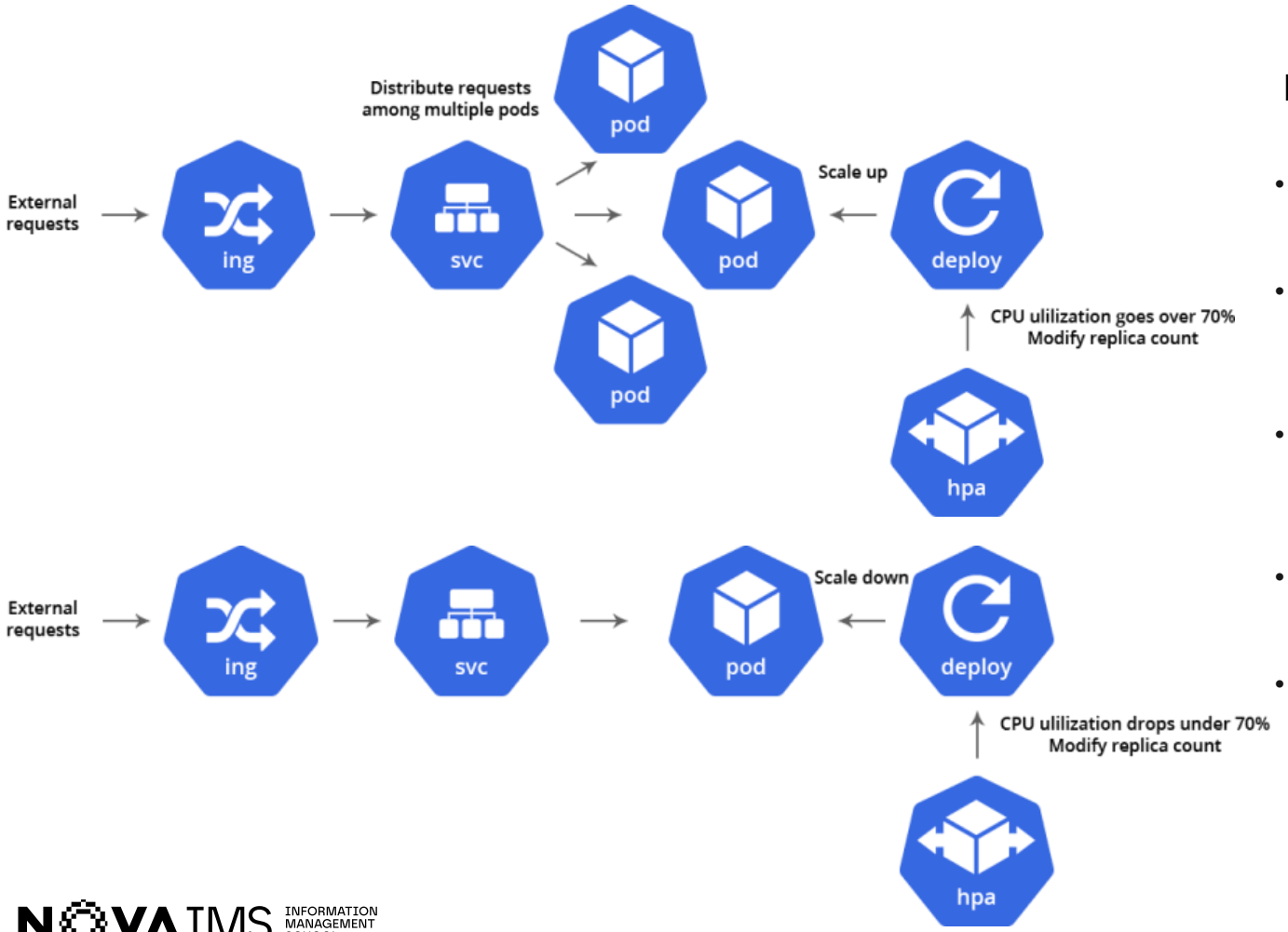
- ❑ As na example, we can have 3GiB node and a 2GiB in stash.
- ❑ When your application starts consuming more or you simply deploy more pods those 2 extra GiB become immediately available.
- ❑ You start to pay for the new resources just for the moment you consume them.

One huge node in a Kubernetes cluster is not a good idea since all deployments will be affected in case a major incident. Moreover several nodes in a stand-by mode is not cost efficient.

03: Model Deployment Requirements

kubernetes final resume

Why scalability is so relevant?



Horizontal auto scaling in a cloud system:

- We have a node with a couple of running pods, and a service starts getting a lot of requests.
- It will scale up for more pods and and the load balancer of Kubernetes will redirect the requests for healthy pods (or our replicas).
- Those new pods will require more resources, so that will be placed on the same node and utilize dynamic RAM, or a new node will be added.
- You can also put some bounds on the resources allocated for the pods.
- If you know that a particular service should not consume more than X GiB, and there's a memory leak if it does, you instruct Kubernetes to kill the pod when RAM utilization reaches XGB. A new pod will start automatically.

03: Model Deployment Requirements

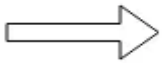
kubernetes final resume

Data Scientists and Kubernetes

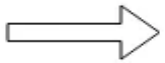
- Wrapping Python machine learning models in containers is how many people start putting initial models into production.
- As far as the complexity of the problem starts to grow, Kubernetes is typically the next step.
- As more machine learning applications are deployed on Kubernetes, this gravity effect tends to have a full orchestration for machine learning as well.

You realize that you are implementing a full multitenant data science platform on Kubernetes from scratch, with schedulers and pipelines.

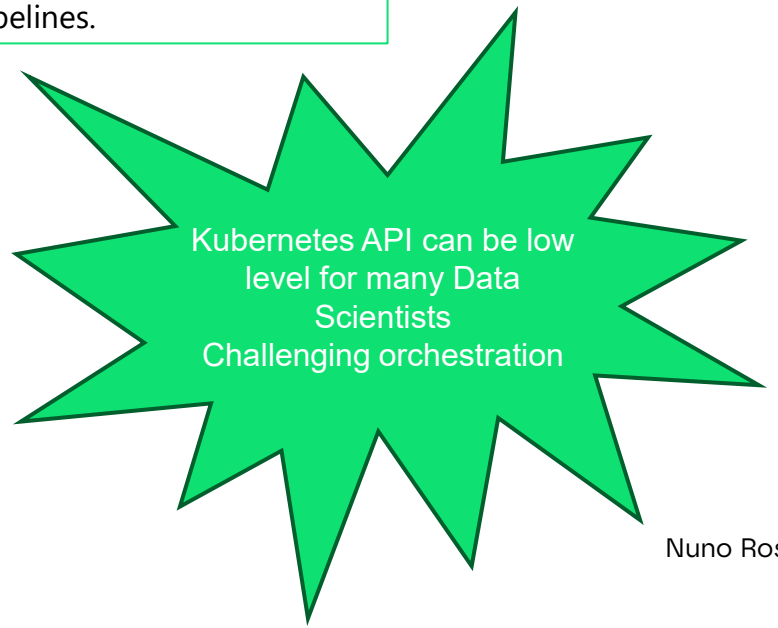
Machine learning model



Docker image



Kubernetes job



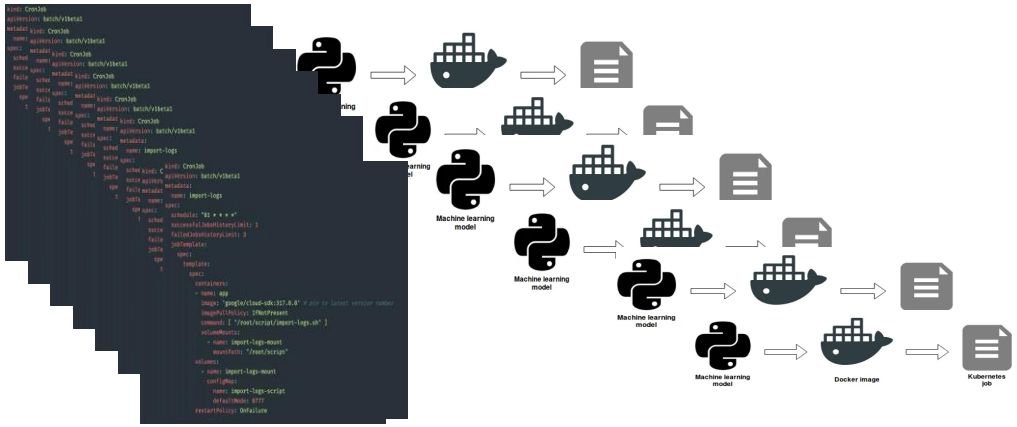
03: Model Deployment Requirements

kubernetes final resume

Data Scientists and Kubernetes

- Wrapping Python machine learning models in containers is how many people start putting initial models into production.
- As far as the complexity of the problem starts to grow, Kubernetes is typically the next step.
- As more machine learning applications are deployed on Kubernetes, this gravity effect tends to have a full orchestration for machine learning as well.

You realize that you are implementing a full multitenant data science platform on Kubernetes from scratch, with schedulers and pipelines.



Kubernetes API can be low level for many Data Scientists
Challenging orchestration

03: Model Deployment Requirements

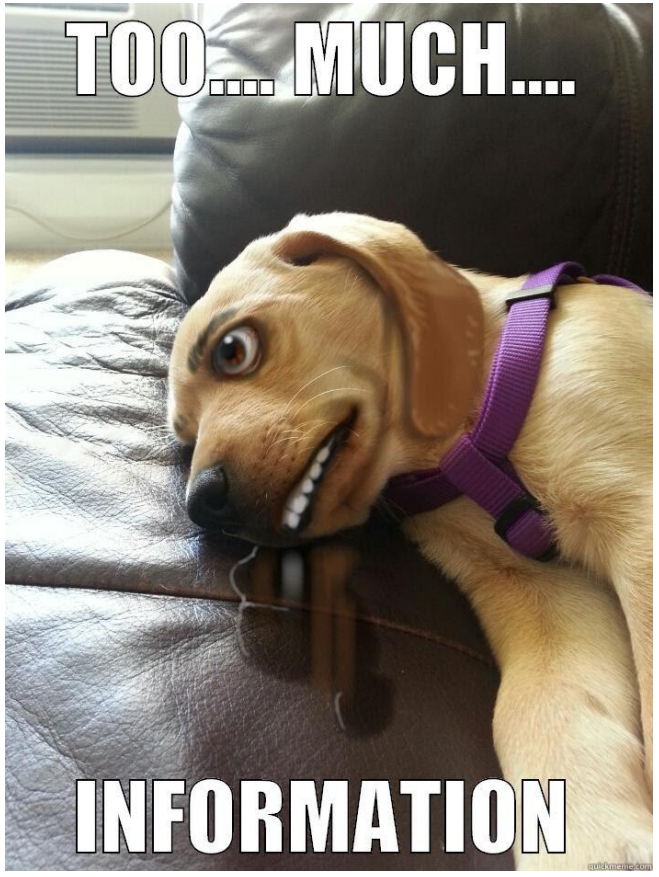
kubernetes final resume

Data Scientists and Kubernetes

- With Kubernetes the complexity of the training process quickly increases when additional steps are needed like data extraction, data preparation, model testing or post-processing.

- A data scientist already has to know a number of techniques and technologies, does not need one more complex problem in the Kubernetes API to the extensive bucket list :)

  - Containers.
  - Packaging.
  - Kubernetes service endpoints.
  - Persistent volumes.
  - Scaling.
  - Immutable deployments.
  - GPUs, Drivers...
  - Cloud APIs
  - DevOps
  - Cronjobs
  - ...



Model Deployment

Kubernetes

02: Model Deployment

Kubeflow

- Kubeflow was conceived of as a way to run machine learning workflows on Kubernetes and is typically used for the following reasons:
  - You want to train/serve machine learning models in different environments with different libraries and scalability.
  - You want to use Jupyter Notebooks to manage machine learning training jobs (not just TensorFlow jobs).
  - You want to launch training jobs that use resources.

Kubeflow

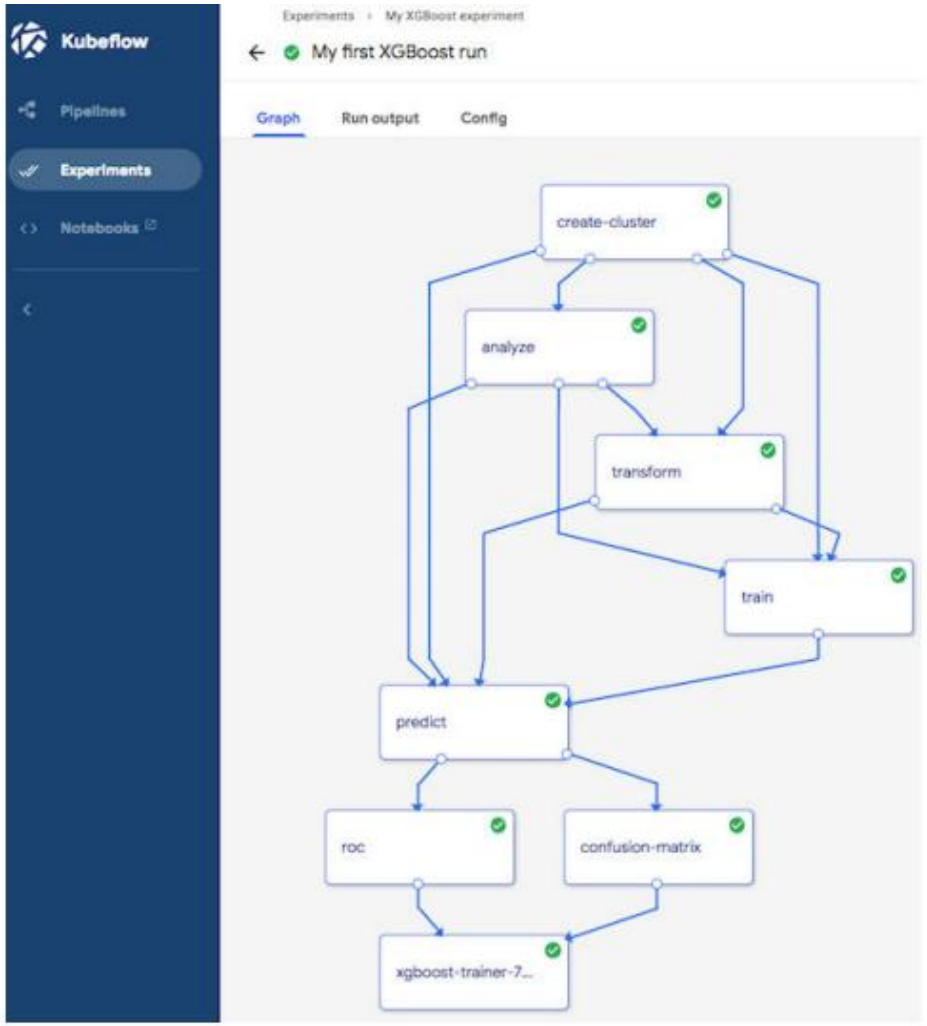
- It consists of a number of different services that give you the tools you need to develop, train, test and deploy models.
- Forget Kubernetes' internals. We currently use services specifically for developing models and to create pipelines for creating and scheduling workflows.

GOAL: Kubeflow tell Kubernetes clusters about specificities and nuances that each execution of a machine learning library has, such as parallel training on multiple Kubernetes nodes.

02: Model Deployment

Kubeflow

Components and services

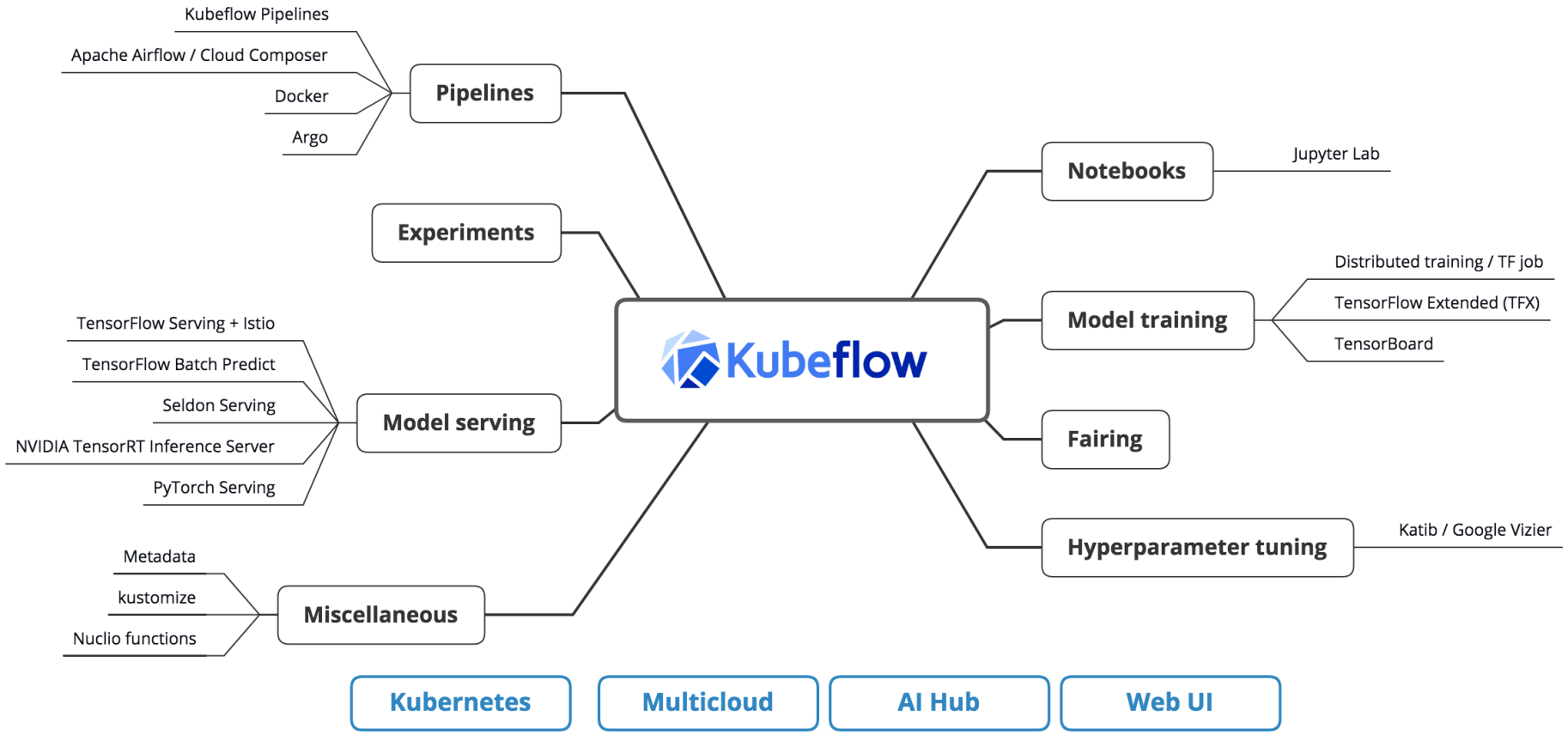


- Like Kedro, a Kubeflow Pipeline is a directed acyclic graph (DAG), with the respective components of the workflow.
- Pipelines define input parameter required to run the pipeline and then show each component's output and how it is wired as input to the next stage in the graph (similar to Kedro).
- The upgrade here is to have an UI to define input parameters per run and then launch the job. We can save several outputs.

02: Model Deployment

Kubeflow

Components and services



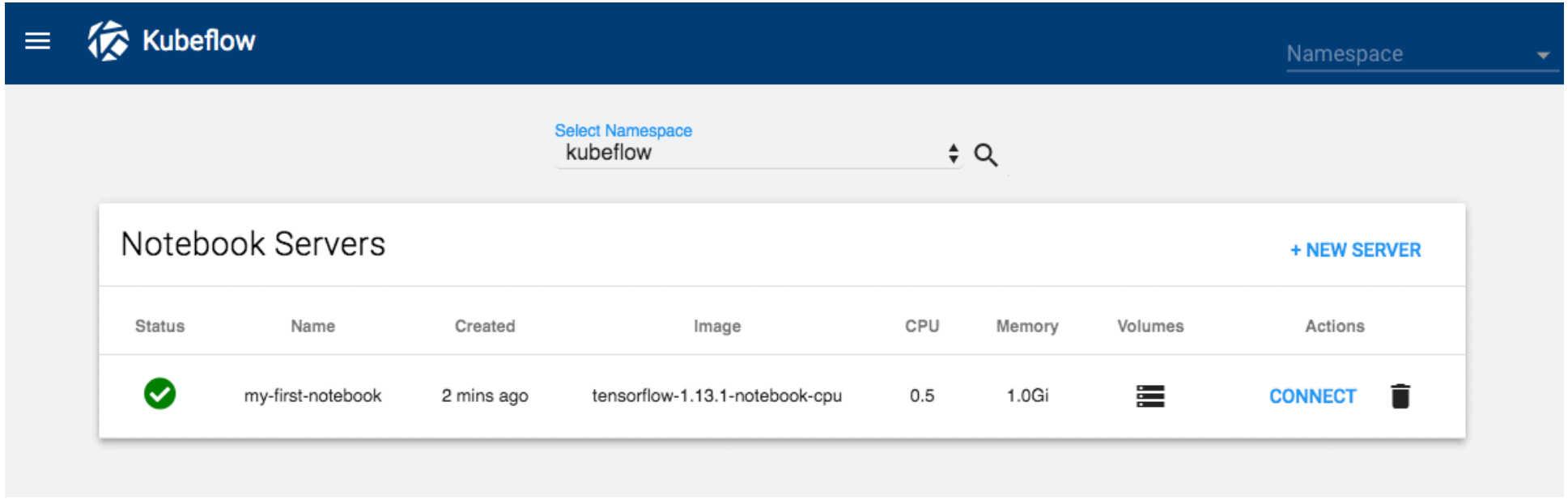
These components work together to provide a scalable and secure system for running machine learning jobs (notebook-based jobs and also jobs outside of notebooks).

02: Model Deployment

Kubeflow

Components and services

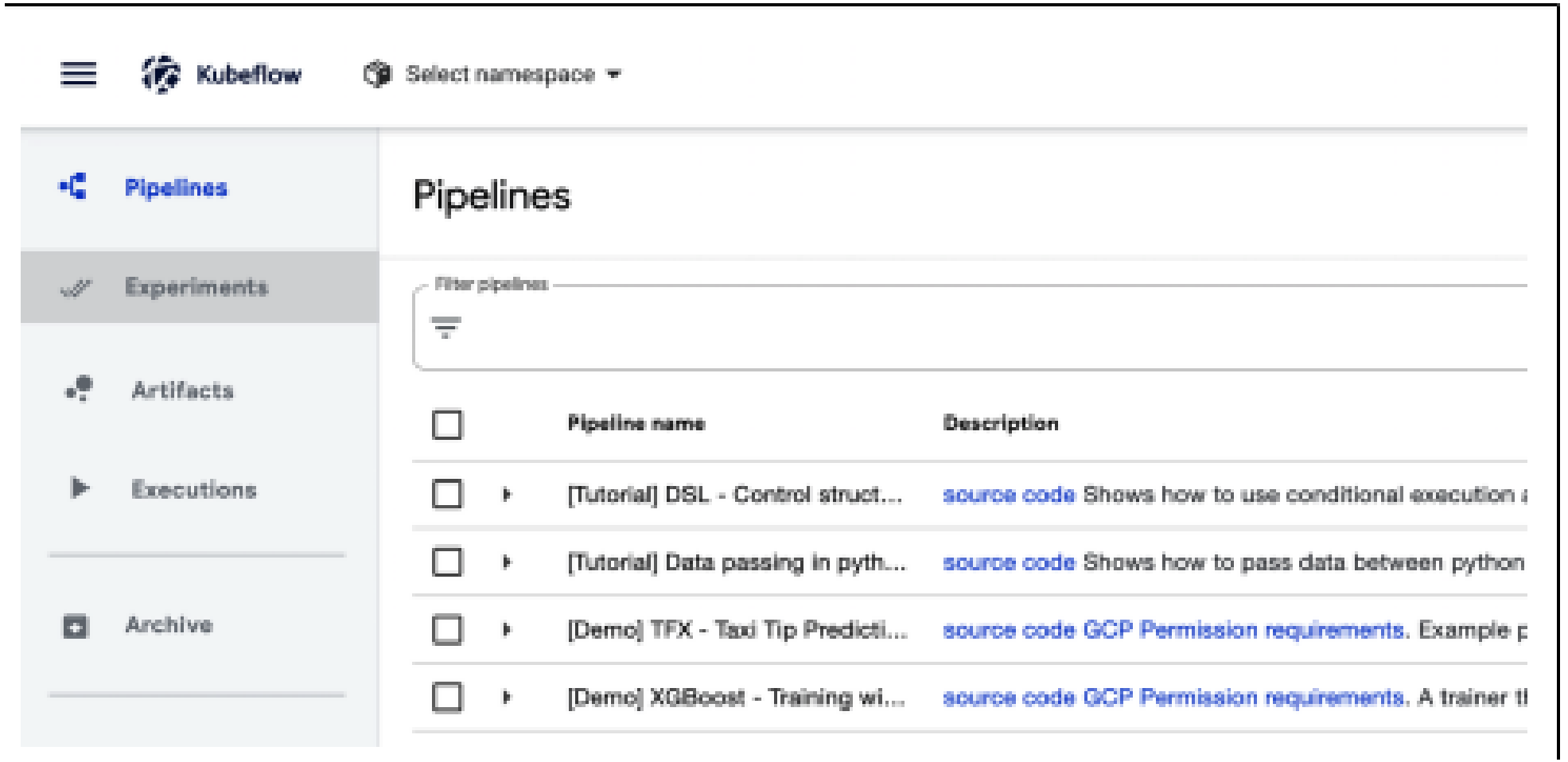
Data Exploration with Notebooks



We can see it as a data exploration inspired by containers, with isolated environment per use case or service!

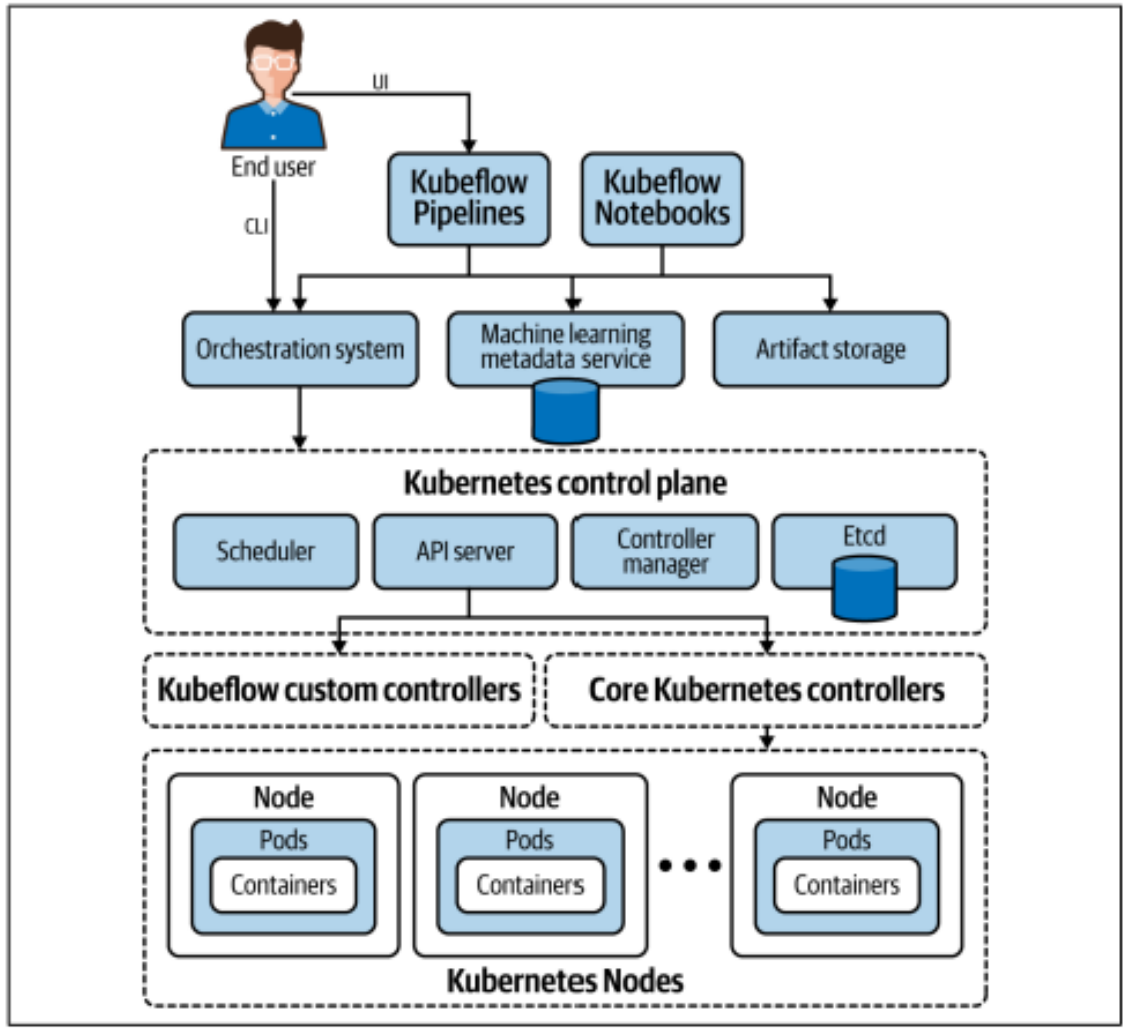
02: Model Deployment

Kubeflow
User Interface



02: Model Deployment

Kubeflow



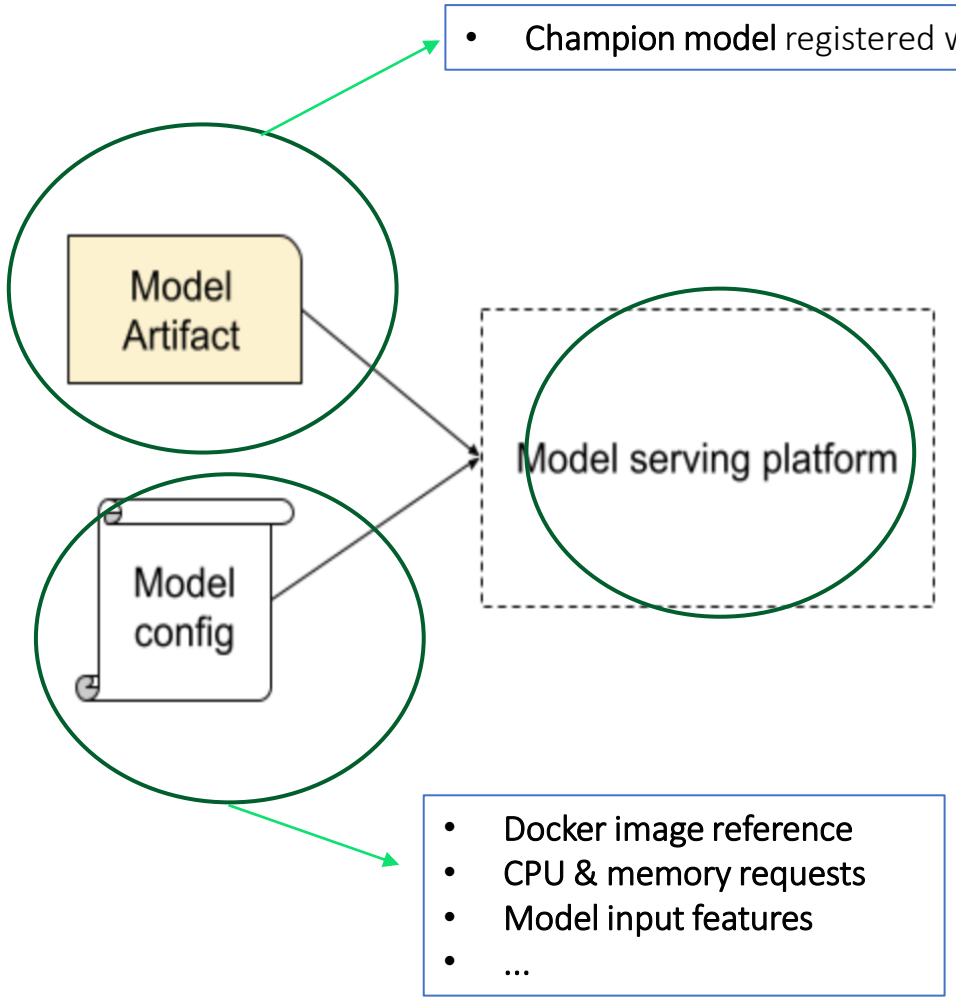
There are three ways to run a machine learning job on Kubeflow:

- **CLI**, we can use kubectl to launch a Python container to run a Python script. This script may (or may not) write metadata back to the Kubeflow system.
- **Kubeflow Pipelines**, we construct a DAG of operations in a configuration file (or via the UI) to schedule a series of orchestrated jobs on Kubeflow.
- **Notebooks**, we launch a notebook server that may run one or more notebooks. From there each notebook can run different code on the same notebook server. Later in this chapter we dig into the architecture of the notebook server launcher and notebook server system.

Model Serving

03: Model Serving

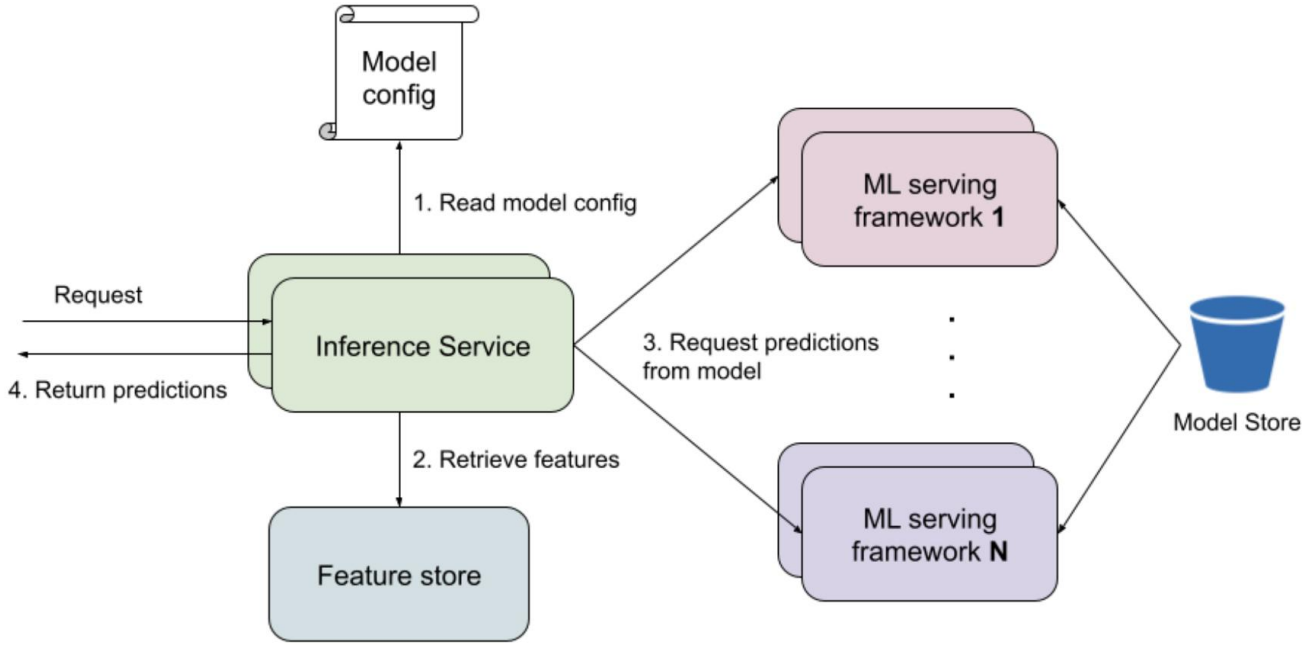
Naive structure



Business Need	Solution
A business needs to infer a single ML model and then as per the demand	Virtual machines using A REST API (Fast API, Flask)
A business needs to infer more models on demand and auto-scale when requests from the users increase in quantity	Containers with Kubernetes and REST API
A business needs a robust and scalable ML model service and to control their business operations	Servless application (Cloud providers)
A business needs to hand continuous requests from the user for multiple ML models and scale as per the quantity of the users to serve their requests	Model streaming platform

03: Model Serving

Ideal structure



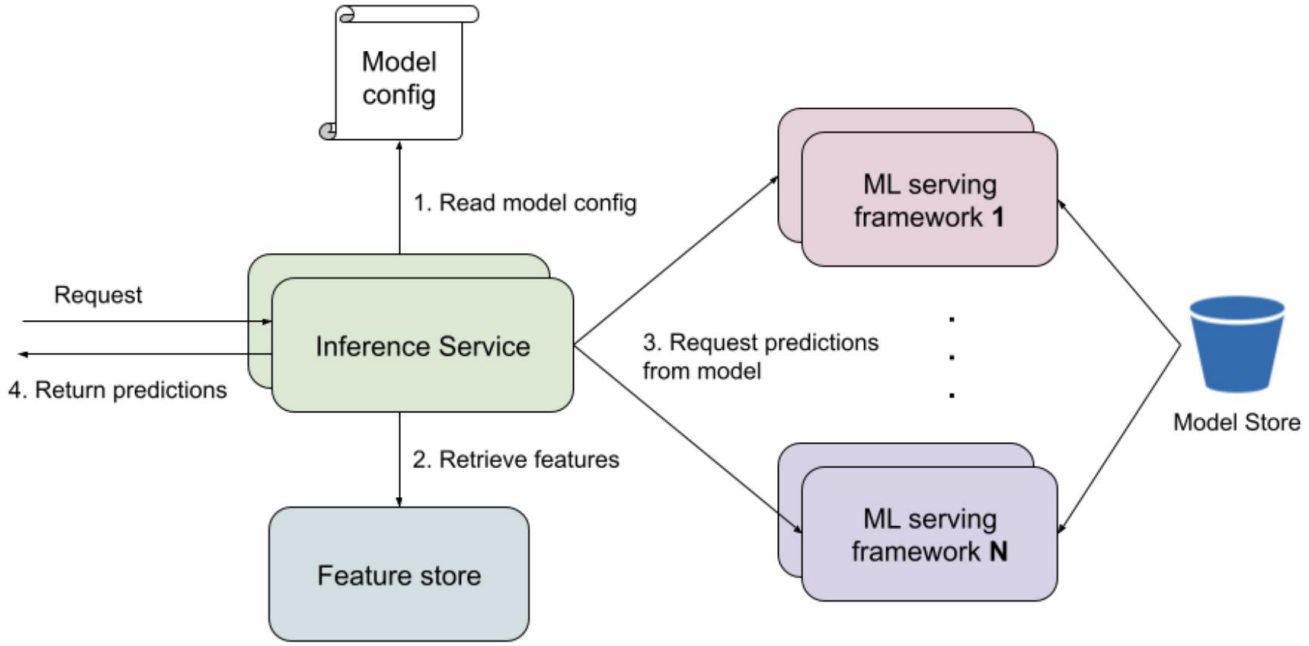
- **Inference Service** - **serving API where clients can send requests to different routes to get predictions from different models**. The main idea is to unify serving logic across models and provides easier interaction with other internal services. The Inference Service can focus on I/O-bound operations while the model serving frameworks focus on compute-bound operations. They can be scaled based on their unique performance characteristics.

- **Feature and Model store** - Storage for features and model (like Hopsworks and MLflow from the classes).

- **ML serving framework** - is a container with all the dependencies required to run a certain type of model. The container may serve one or more models. To ensure reusable images, models usually aren't baked into the container image. Rather, the container loads in models when it starts, or when a request for that model comes in.

03: Model Serving

Ideal structure

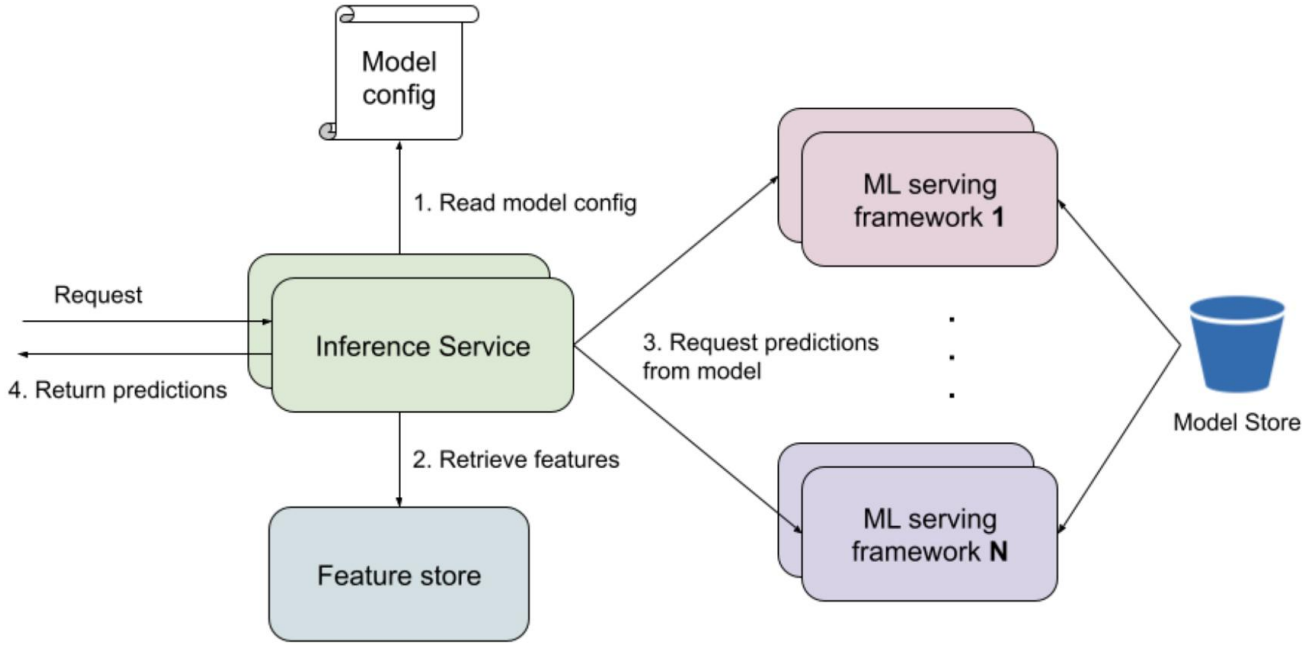


Simplest way to serve:

- One model per set of ML serving framework containers.
- The fraud model is in a set of XGBoost containers, the pricing model is in a set of TorchServe containers, and the recommender model is in another set of TorchServe containers.
- With a **good container orchestration framework (kubernetes), it is possible to scale** it to even more models.

03: Model Serving

Ideal structure



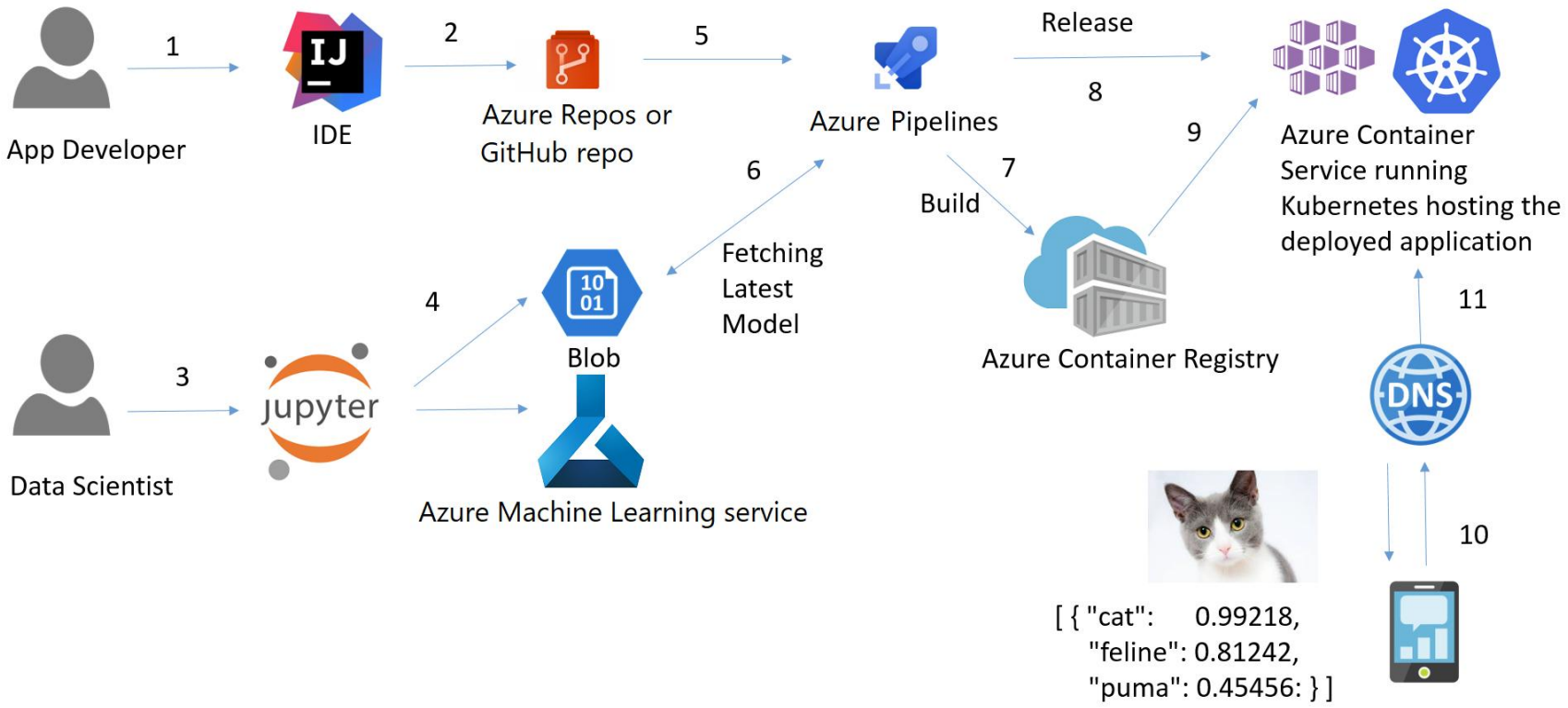
Over time, it will exist more models and multiple versions of the same model type, which results in many containers - **moving towards serving multiple models per container**:

- **You want higher resource utilization**. If the pricing model is idle, there are still 2 or more running containers to make sure the model is available and take up the requested number of resources. If a TorchServe container serves both the recommender and pricing models, you can use a higher percentage of resources. The tradeoff is you can have a situation where one model is slowed down because the other is competing for resources.

- L**imit on the number of containers**. Kubernetes has a per cluster limit, or the cluster is shared and there's a limit on the number of containers for the serving platform. It can enter in the budget management layer and resources.

03: Model Serving

Final picture



Model Serving: The software that wraps the model to answer queries (e.g., FastAPI, MLflow). Next week.

Model Deployment: The infrastructure that hosts the served model, manages traffic, and ensures uptime (e.g., Kubernetes, Azure, Load Balancers, Docker).

03: Model Serving

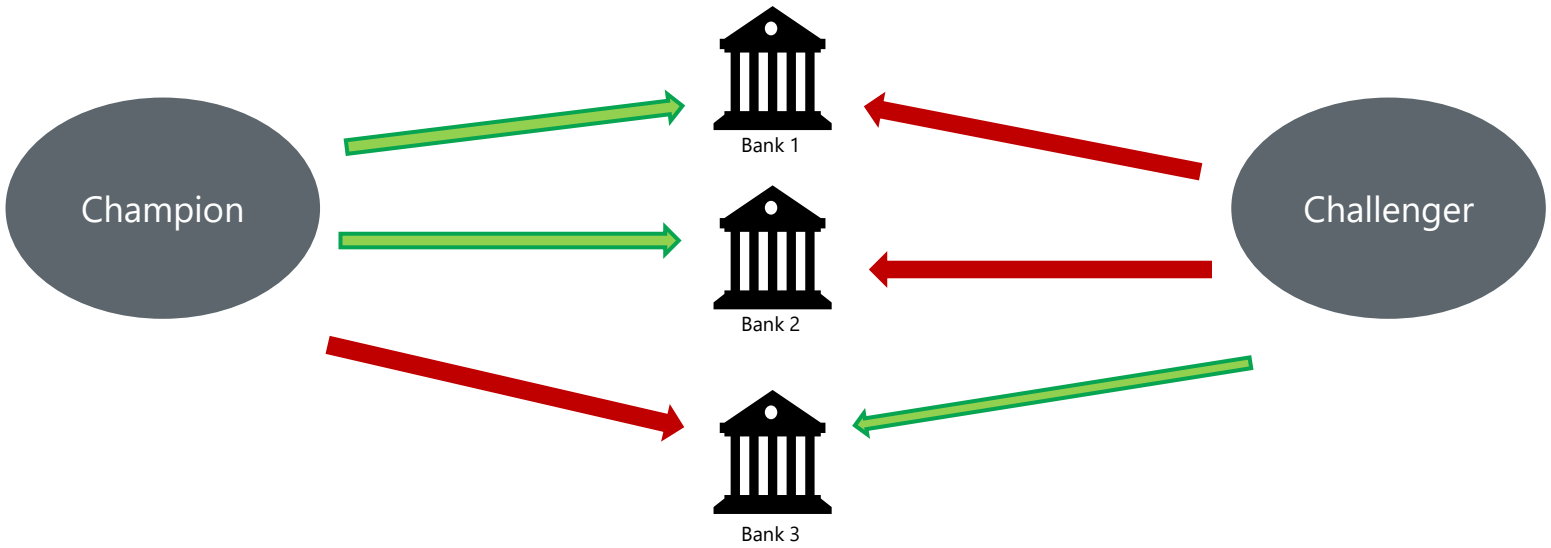
Real-Time vs. Batch

	Real-Time (Online) Inference	Batch (Offline) Inference
What is it?	Predicting one observation at a time, instantly.	Predicting millions of observations all at once.
The Trigger	A user action (e.g., clicking "checkout").	A schedule (e.g., 2:00 AM every Sunday).
Latency Requirement	**Milliseconds** (Speed is critical).	**Hours** (Throughput is critical).
Infrastructure Focus	High availability, APIs (FastAPI), Load Balancers.	Heavy compute, parallel processing, big data storage.
Banking Example	Credit Card Fraud Detection (Must block the card *now*).	Monthly Churn Prediction (Scoring all customers overnight).

03: Model Serving

Final remarks on model deployment

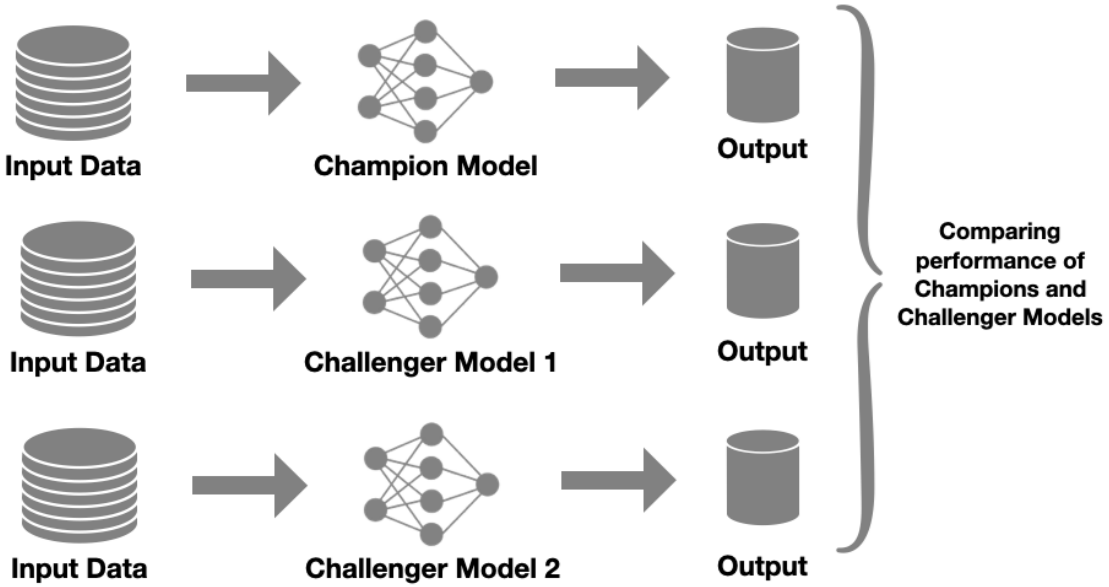
- When sending a new model version to production, we want to avoid downtime for real time scoring. **The basic idea is that a new system can be set up next to the stable one, and when it's functional, the workload can be directed to the newly deployed version deployment** (Kubernetes handles this natively).
- Another more advanced solution to mitigate the risk is **to have canary releases (also called canary deployments)**. The idea is that the stable version of the model is kept in production, but **a certain percentage of the workload is redirected to the new model, and results are monitored.**



Canary deployments happen in production to limit the 'blast radius' of potential failures. If a model rolled out to a small group of banks encounters an issue, the rest of the system is protected.

03: Model Serving

Shadow testing



Advantages:

- The same input data is used for comparing the performance of the Champion model and various Challenger Models. Therefore, there is no room for sampling bias to originate.
- It is safer as the predictions from Challenger models are not being used yet.

Disadvantages:

- Multiple models are used for inferencing on the same data, this results in an increase in the consumption of resources such as memory, processing power, storage, etc, which indirectly results in an increase in cost. Hence, shadow testing is more expensive.

It is **absolutely mandatory** that:

1. When the challenger model encounters an unexpected issue and fails, the production environment will not experience any discontinuation or degradation in terms of response time.
2. Actions taken by the system depend only on the prediction of the champion model, and they happen only once. For example, in a fraud detection use case, imagine that by mistake the challenger model is plugged directly into the system, charging each transaction twice—a catastrophic scenario.

Is there any scenario where shadow testing is not possible?

03: Model Serving

Shadow testing

Shadow testing is not possible when:

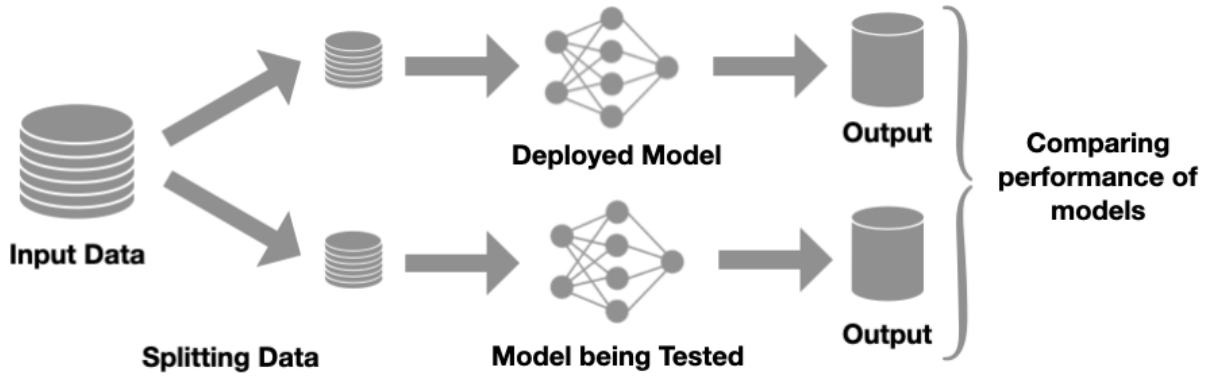
1. The ground truth cannot be evaluated for both models. For an insurance fraud detection model, if to obtain the ground truth, we have to contact all clients, then we have to do it for the positive predictions of two models; it would increase the workload too much, because some frauds are only detected by only one model. As a result, randomly applying only the new. model to a small fraction of the requests will allow the workload to remain constant. *

2. The objective to optimize is only indirectly related to the performance of the prediction. Imagine an ad engine based on an ML model that predicts if a user will click on the ad. Now imagine that it is evaluated on the buy rate, i.e., whether the user bought the product or service.

*Another example is a recommendation engine where the prediction gives a list of items on which a given customer is likely to click if there are presented. So it will be impossible to know what would regarding to items that were not presented to the client.

03: Model Serving

A/B testing



Advantages:

- The total number of times a prediction needs to be made does not change, hence the resources and the cost are lower.

Disadvantages:

- The data on which the predictions are being made is different for the models. Efforts should be made to ensure that the split in the data does not cause any sampling bias for testing the Machine Learning models.
- It is often more difficult to implement than Shadow Testing.
- Since the predictions made by the model being tested are being used, there is a risk involved if the performance of the model being tested degrades considerably between initial testing and testing in production.

Statistical protocol:
- The resulting metrics are compared using statistical tests, and the null hypothesis is either rejected or retained.
- Define beforehand the sample size for the desired minimum effect size, which is the minimum difference between the two models' performance metrics.
- Fix a test duration.